H6 and H7 Analysis

Team BRSM Team BNT

```
get_time_of_day <- function(ts) {
  time_ist <- as_datetime(ts / 1000, tz = "Asia/Kolkata")
  hour <- hour(time_ist)
  case_when(
    hour >= 6  & hour < 16 ~ "Day time",
    # hour >= 12 & hour < 17 ~ "Afternoon",
    hour >= 16 & hour < 20 ~ "Evening",
    TRUE ~ "Night"
  )
}
prepare_common_columns <- function(df) {
  df %>%
    mutate(
      Accuracy.IR = suppressWarnings(as.numeric(Accuracy.IR)),
      Accuracy.WR = suppressWarnings(as.numeric(Accuracy.WR)),
      Reaction_time_IR = suppressWarnings(as.numeric(na_if(Reaction_time_IR, "N/A"))),
      Reaction_time_WR = suppressWarnings(as.numeric(na_if(Reaction_time_WR, "N/A")))
    )
}

safe_normality_tests <- function(x) {
  x <- x[is.finite(x)]
  n <- length(x)
  s <- if (n > 1) sd(x) else NA_real_

  if (n < 3 || is.na(s) || s == 0) {
    return(tibble(
      n = n,
      AD_stat = NA_real_,
      AD_p = NA_real_,
      normality_note = "Insufficient variability/sample size"
    ))
  }

  ad_out <- tryCatch(ad.test(x), error = function(e) NULL)

  tibble(
    n = n,
    AD_stat = if (!is.null(ad_out)) unname(ad_out$statistic) else NA_real_,
    AD_p = if (!is.null(ad_out)) ad_out$p.value else NA_real_,
    normality_note = case_when(
      n < 25 ~ "Small sample: use AD with QQ interpretation",
      TRUE ~ "Use QQ + AD jointly"
    )
```

```
  )
}

wilcox_effect_size_r <- function(wilcox_obj, n_pairs) {
  if (is.null(wilcox_obj$statistic) || n_pairs <= 0) return(NA_real_)
  w <- as.numeric(wilcox_obj$statistic)
  mean_w <- n_pairs * (n_pairs + 1) / 4
  sd_w <- sqrt(n_pairs * (n_pairs + 1) * (2 * n_pairs + 1) / 24)
  if (sd_w == 0) return(NA_real_)
  z <- (w - mean_w) / sd_w
  abs(z) / sqrt(n_pairs)
}

kruskal_epsilon_sq <- function(kruskal_obj, n_total, k_groups) {
  if (is.null(kruskal_obj$statistic) || n_total <= k_groups) return(NA_real_)
  h <- as.numeric(kruskal_obj$statistic)
  (h - k_groups + 1) / (n_total - k_groups)
}

summarise_phase_metrics <- function(df) {
  df %>%
    arrange(participant_ID, Timestamp) %>%
    group_by(participant_ID, Phase) %>%
    mutate(prev_IR = lag(Accuracy.IR)) %>%
    summarise(
      # Treat "novel" as either NA-coded or explicit FALSE-coded non-repeats.
      # This guards against schema differences across log exports.
      Novel_Trials = sum((is.na(isRepeat) | isRepeat == FALSE) & Event == "Sentence shown", na.rm = TRU
      Total_Repeats = sum(isRepeat == TRUE & Event == "Sentence shown", na.rm = TRUE),
      Hits = sum(Event == "IR pressed" & isRepeat == TRUE & Accuracy.IR == 1, na.rm = TRUE),
      False_Alarms = sum(Event == "IR pressed" & (is.na(isRepeat) | isRepeat == FALSE), na.rm = TRUE),
      Hit_Rate = if_else(Total_Repeats > 0, Hits / Total_Repeats, 0),
      False_Alarm_Rate = if_else(Novel_Trials > 0, False_Alarms / Novel_Trials, 0),
      IR_Memorability = Hit_Rate - False_Alarm_Rate,
      # WR is conditioned on correctly recognized IR events by task design.
      WR_correct = sum(Event == "WR pressed" & Accuracy.WR == 1 & prev_IR == 1, na.rm = TRUE),
      WR_total = sum(Event == "WR pressed" & prev_IR == 1, na.rm = TRUE),
      WR_Accuracy = if_else(WR_total > 0, WR_correct / WR_total, 0),
      Avg_RT_IR = mean(Reaction_time_IR, na.rm = TRUE),
      Avg_RT_WR = mean(Reaction_time_WR, na.rm = TRUE),
      First_Timestamp = min(Timestamp, na.rm = TRUE),
      Avg_Timestamp = mean(Timestamp, na.rm = TRUE),
      Sentence_Shown_Count = sum(Event == "Sentence shown", na.rm = TRUE),
      .groups = "drop"
    )
}

extract_phase_data <- function(file_path) {
  df <- read_csv(file_path, show_col_types = FALSE)
  colnames(df) <- make.names(colnames(df))
  df <- prepare_common_columns(df)

  practice_indices <- which(str_detect(df$Event, "^Practice "))
```

```
rest_indices <- which(df$Event == "Rest Phase started")

if (length(practice_indices) == 0 || length(rest_indices) < 2) {
  return(NULL)
}

practice_end <- max(practice_indices)

practice_data <- df[seq_len(practice_end), ] %>%
  mutate(
    Phase = "Practice",
    Event = str_remove(Event, "^Practice ")
  ) %>%
  filter(
    isTarget == TRUE,
    Event %in% c("Sentence shown", "IR pressed", "WR pressed", "Missed"),
    Event != "gap_time",
    participant_ID != "N/A",
    Timestamp != "N/A",
    Button %in% c("Yes", "No", "None", "N/A", "Spacebar")
  ) %>%
  filter(
    (is.na(Reaction_time_IR) | (Reaction_time_IR >= 150 & Reaction_time_IR <= 3000)) &
    (is.na(Reaction_time_WR) | (Reaction_time_WR >= 150 & Reaction_time_WR <= 3000))
  )

block1 <- df[(practice_end + 1):(rest_indices[1] - 1), ] %>% mutate(Block_Number = 1)
block2 <- df[(rest_indices[1] + 1):(rest_indices[2] - 1), ] %>% mutate(Block_Number = 2)
block3 <- df[(rest_indices[2] + 1):nrow(df), ] %>% mutate(Block_Number = 3)
blocks <- list(block1, block2, block3)

valid_blocks <- list()

for (block in blocks) {
  target_trials <- block %>% filter(isTarget == TRUE, Event == "Sentence shown")

  if (nrow(target_trials) != 32) {
    next
  }

  validation_trials <- block %>% filter(isValidation == TRUE)
  correct_val <- sum(validation_trials$isRepeat == TRUE & validation_trials$Accuracy.IR == 1, na.rm =
  wrong_val <- sum(validation_trials$isRepeat == TRUE & validation_trials$Accuracy.IR == 0, na.rm = TI
  repeat_trials <- validation_trials %>% filter(Event == "Sentence shown", isRepeat == TRUE)
  ir_presses <- validation_trials %>% filter(Event == "IR pressed", isRepeat == TRUE)
  missed_val <- nrow(repeat_trials) - nrow(ir_presses)

  if (correct_val > (wrong_val / 2) + missed_val) {
    valid_blocks <- append(valid_blocks, list(block))
  }
}

if (length(valid_blocks) == 0) {
```

```
    return(NULL)
  }

  main_data <- bind_rows(valid_blocks) %>%
    mutate(Phase = "Main") %>%
    filter(
      isTarget == TRUE,
      Event != "gap_time",
      participant_ID != "N/A",
      Timestamp != "N/A",
      Button %in% c("Yes", "No", "None", "N/A", "Spacebar")
    ) %>%
    filter(
      (is.na(Reaction_time_IR) | (Reaction_time_IR >= 150 & Reaction_time_IR <= 3000)) &
      (is.na(Reaction_time_WR) | (Reaction_time_WR >= 150 & Reaction_time_WR <= 3000))
    )

  bind_rows(practice_data, main_data)
}
```

```
log_files <- list.files("dataset", pattern = "\\.log$", full.names = TRUE)

phase_level_dataset <- log_files %>%
  lapply(extract_phase_data) %>%
  bind_rows()

participant_phase_metrics <- summarise_phase_metrics(phase_level_dataset) %>%
  mutate(
    Phase = factor(Phase, levels = c("Practice", "Main")),
    Time_of_Day = get_time_of_day(First_Timestamp),
    Time_of_Day = factor(Time_of_Day, levels = c("Day time", "Evening", "Night"))
  )

# Minimum number of "Sentence shown" rows retained *per phase* for analysis.
# IR/WR metrics are proportions over these trials - very few sentences very noisy estimates.
# Practice is short in the protocol (often ~10); Main stacks validated blocks (often 32 per block).
min_sentences_practice <- 10L
min_sentences_main <- 32L

participant_phase_metrics <- participant_phase_metrics %>%
  mutate(
    min_required = if_else(Phase == "Practice", min_sentences_practice, min_sentences_main),
    meets_sentence_threshold = Sentence_Shown_Count >= min_required
  )

participant_phase_metrics
```

```
## # A tibble: 228 x 20
##    participant_ID Phase   Novel_Trials Total_Repeats  Hits False_Alarms Hit_Rate
##             <dbl> <fct>          <int>         <int> <int>        <int>    <dbl>
##  1            232 Main              48            48    47            0    0.979
##  2            232 Practi~           10            10     9            1    0.9
##  3            235 Main              48            48    36            0    0.75
```

```
##  4           235 Practi~           5            5     4           0    0.8
##  5           236 Main             48           48    44           0    0.917
##  6           236 Practi~           5            5     5           0    1
##  7           241 Main             48           48    40           3    0.833
##  8           241 Practi~           5            5     5           0    1
##  9           242 Main             48           48    37           1    0.771
## 10           242 Practi~           5            5     3           0    0.6
## # i 218 more rows
## # i 13 more variables: False_Alarm_Rate <dbl>, IR_Memorability <dbl>,
## #   WR_correct <int>, WR_total <int>, WR_Accuracy <dbl>, Avg_RT_IR <dbl>,
## #   Avg_RT_WR <dbl>, First_Timestamp <dbl>, Avg_Timestamp <dbl>,
## #   Sentence_Shown_Count <int>, Time_of_Day <fct>, min_required <int>,
## #   meets_sentence_threshold <lgl>
```

```r
# Rows excluded by phase (before requiring paired H6 data)
participant_phase_metrics %>%
  count(Phase, meets_sentence_threshold, name = "n_rows")
```

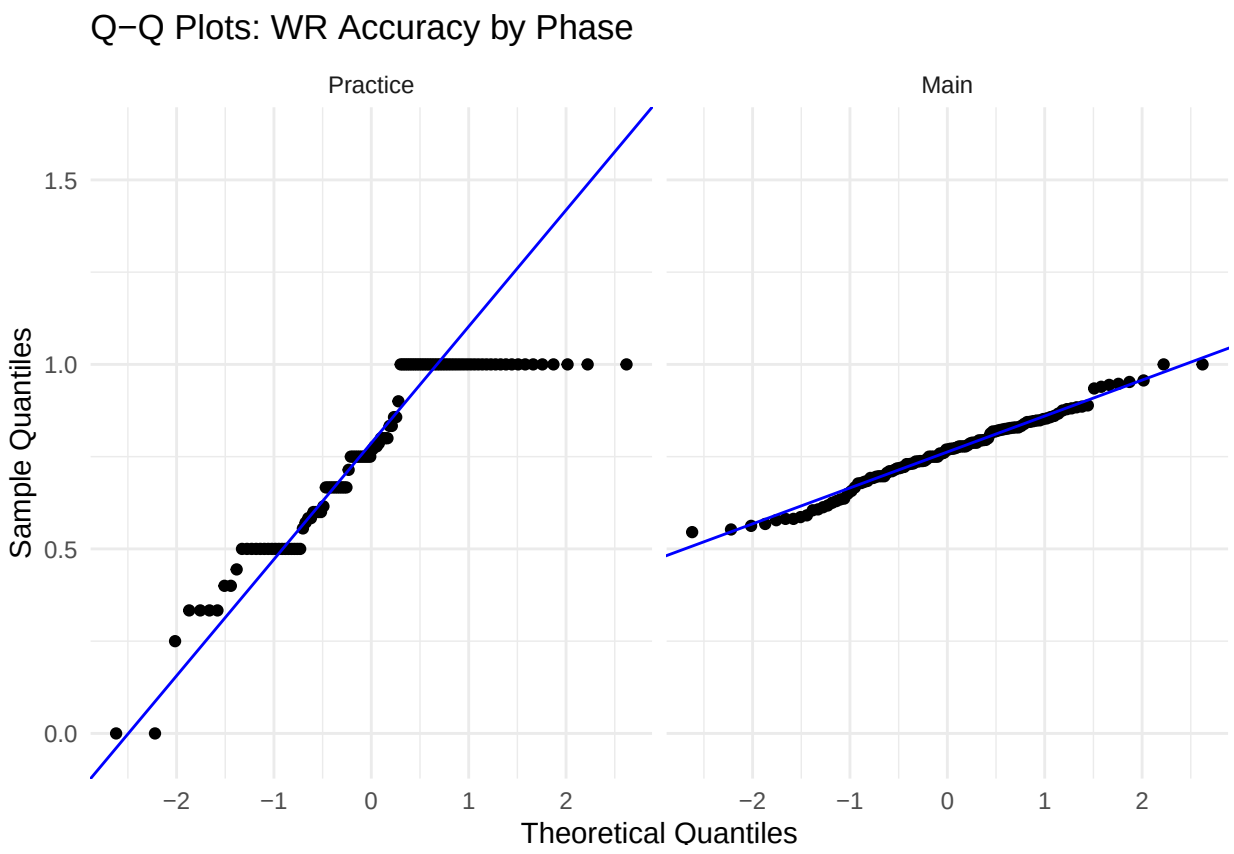
```
## # A tibble: 2 x 3
##   Phase    meets_sentence_threshold n_rows
##   <fct>    <lgl>                       <int>
## 1 Practice TRUE                          114
## 2 Main     TRUE                          114
```

```r
# Keep only participants who contribute both practice and main data for H6,
# and who meet the sentence-count threshold in *each* phase.
h6_data <- participant_phase_metrics %>%
  filter(meets_sentence_threshold) %>%
  group_by(participant_ID) %>%
  filter(n_distinct(Phase) == 2) %>%
  ungroup()

h6_counts <- h6_data %>%
  count(Phase)

h6_counts
```

```
## # A tibble: 2 x 2
##   Phase        n
##   <fct>    <int>
## 1 Practice   114
## 2 Main       114
```

```r
# Normality checks for H6 measures
ggplot(h6_data, aes(sample = IR_Memorability)) +
  stat_qq() +
  stat_qq_line(color = "red") +
  facet_wrap(~ Phase, ncol = 2) +
  labs(
    title = "Q-Q Plots: IR Memorability by Phase",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_minimal()
```


```r
ggplot(h6_data, aes(sample = WR_Accuracy)) +
  stat_qq() +
  stat_qq_line(color = "blue") +
  facet_wrap(~ Phase, ncol = 2) +
  labs(
    title = "Q-Q Plots: WR Accuracy by Phase",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_minimal()
```

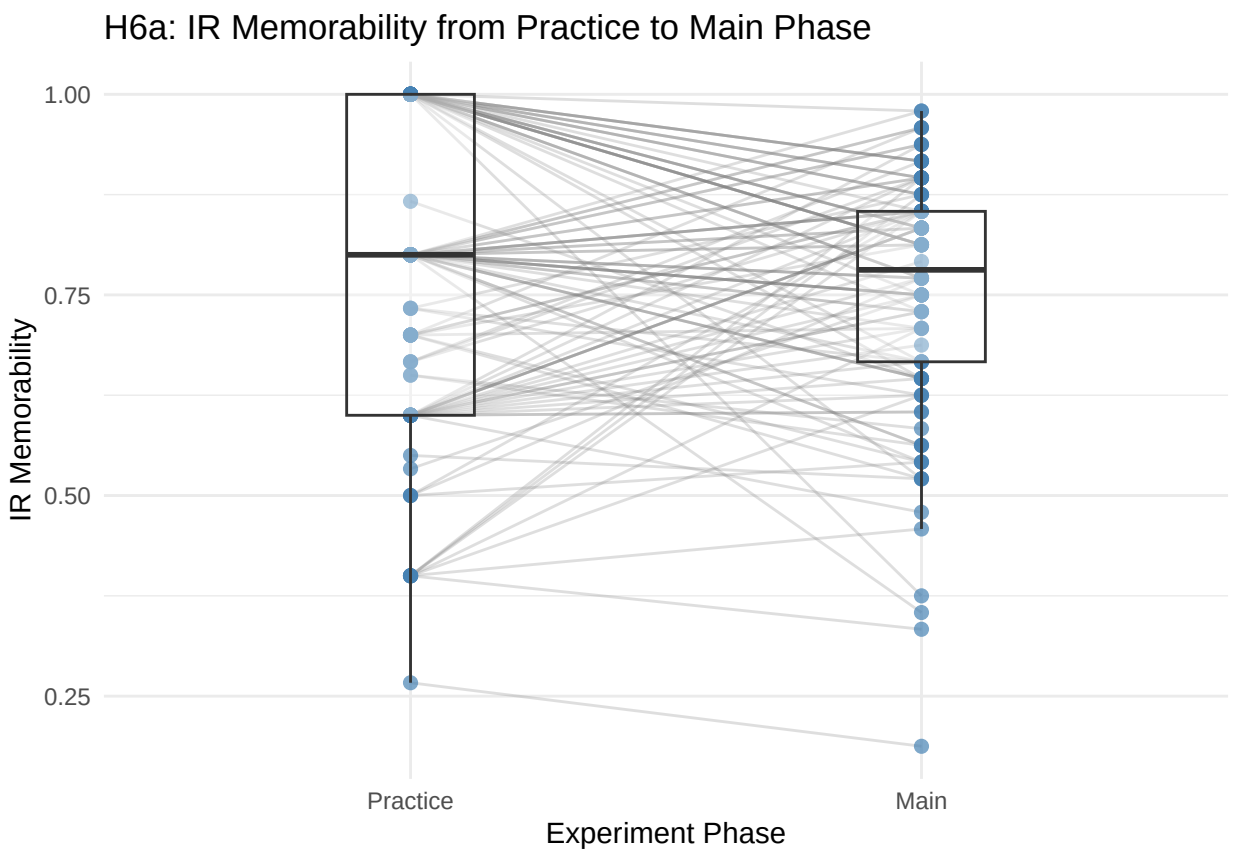



```
h6_data %>%
  group_by(Phase) %>%
  reframe(
    Measure = c("IR_Memorability", "WR_Accuracy"),
    bind_rows(
      safe_normality_tests(IR_Memorability),
      safe_normality_tests(WR_Accuracy)
    )
  )
```

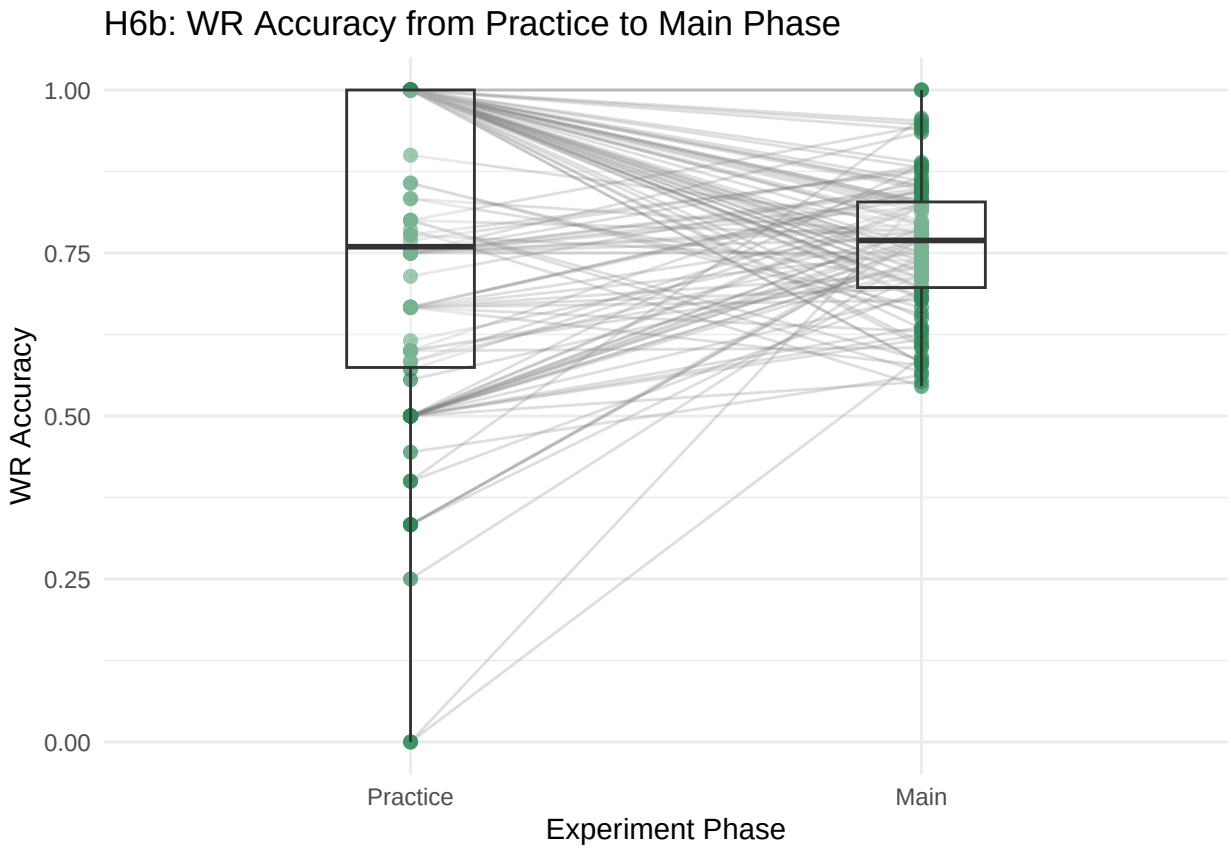
```
## # A tibble: 4 x 6
##   Phase    Measure              n AD_stat     AD_p normality_note
##   <fct>    <chr>            <int>   <dbl>    <dbl> <chr>
## 1 Practice IR_Memorability    114   3.76  1.93e- 9 Use QQ + AD jointly
## 2 Practice WR_Accuracy        114   5.03  1.60e-12 Use QQ + AD jointly
## 3 Main     IR_Memorability    114   2.36  5.23e- 6 Use QQ + AD jointly
## 4 Main     WR_Accuracy        114   0.374 4.11e- 1 Use QQ + AD jointly
```

```
# H6a: IR memorability in practice vs main
ggplot(h6_data, aes(x = Phase, y = IR_Memorability, group = participant_ID)) +
  geom_line(alpha = 0.25, color = "gray50") +
  geom_point(alpha = 0.7, color = "steelblue", size = 2) +
  geom_boxplot(aes(group = Phase), width = 0.25, outlier.shape = NA, alpha = 0.35) +
  labs(
    title = "H6a: IR Memorability from Practice to Main Phase",
    x = "Experiment Phase",
```

```
    y = "IR Memorability"
  ) +
  theme_minimal()
```




```
# H6b: WR accuracy in practice vs main
ggplot(h6_data, aes(x = Phase, y = WR_Accuracy, group = participant_ID)) +
  geom_line(alpha = 0.25, color = "gray50") +
  geom_point(alpha = 0.7, color = "seagreen", size = 2) +
  geom_boxplot(aes(group = Phase), width = 0.25, outlier.shape = NA, alpha = 0.35) +
  labs(
    title = "H6b: WR Accuracy from Practice to Main Phase",
    x = "Experiment Phase",
    y = "WR Accuracy"
  ) +
  theme_minimal()
```




```
h6_wide <- h6_data %>%
  select(participant_ID, Phase, IR_Memorability, WR_Accuracy) %>%
  pivot_wider(
    names_from = Phase,
    values_from = c(IR_Memorability, WR_Accuracy)
  ) %>%
  drop_na()

wilcox_h6_ir <- wilcox.test(
  h6_wide$IR_Memorability_Practice,
  h6_wide$IR_Memorability_Main,
  paired = TRUE,
  exact = FALSE,
  alternative = "less"
)

wilcox_h6_wr <- wilcox.test(
  h6_wide$WR_Accuracy_Practice,
  h6_wide$WR_Accuracy_Main,
  paired = TRUE,
  exact = FALSE,
  alternative = "less"
)

h6_effect_sizes <- tibble(
  Measure = c("H6a_IR_Memorability", "H6b_WR_Accuracy"),
```

```
  n_pairs = nrow(h6_wide),
  wilcox_r = c(
    wilcox_effect_size_r(wilcox_h6_ir, nrow(h6_wide)),
    wilcox_effect_size_r(wilcox_h6_wr, nrow(h6_wide))
  )
)

print("Wilcoxon signed-rank test for H6a (IR Memorability):")
```

```
## [1] "Wilcoxon signed-rank test for H6a (IR Memorability):"
```

```
print(wilcox_h6_ir)
```

```
##
##  Wilcoxon signed rank test with continuity correction
##
## data:  h6_wide$IR_Memorability_Practice and h6_wide$IR_Memorability_Main
## V = 3464, p-value = 0.7015
## alternative hypothesis: true location shift is less than 0
```

```
print("Wilcoxon signed-rank test for H6b (WR Accuracy):")
```

```
## [1] "Wilcoxon signed-rank test for H6b (WR Accuracy):"
```

```
print(wilcox_h6_wr)
```

```
##
##  Wilcoxon signed rank test with continuity correction
##
## data:  h6_wide$WR_Accuracy_Practice and h6_wide$WR_Accuracy_Main
## V = 3191.5, p-value = 0.5976
## alternative hypothesis: true location shift is less than 0
```

```
print("H6 effect sizes (r = |Z|/sqrt(N_pairs)):")
```

```
## [1] "H6 effect sizes (r = |Z|/sqrt(N_pairs)):"
```

```
print(h6_effect_sizes)
```

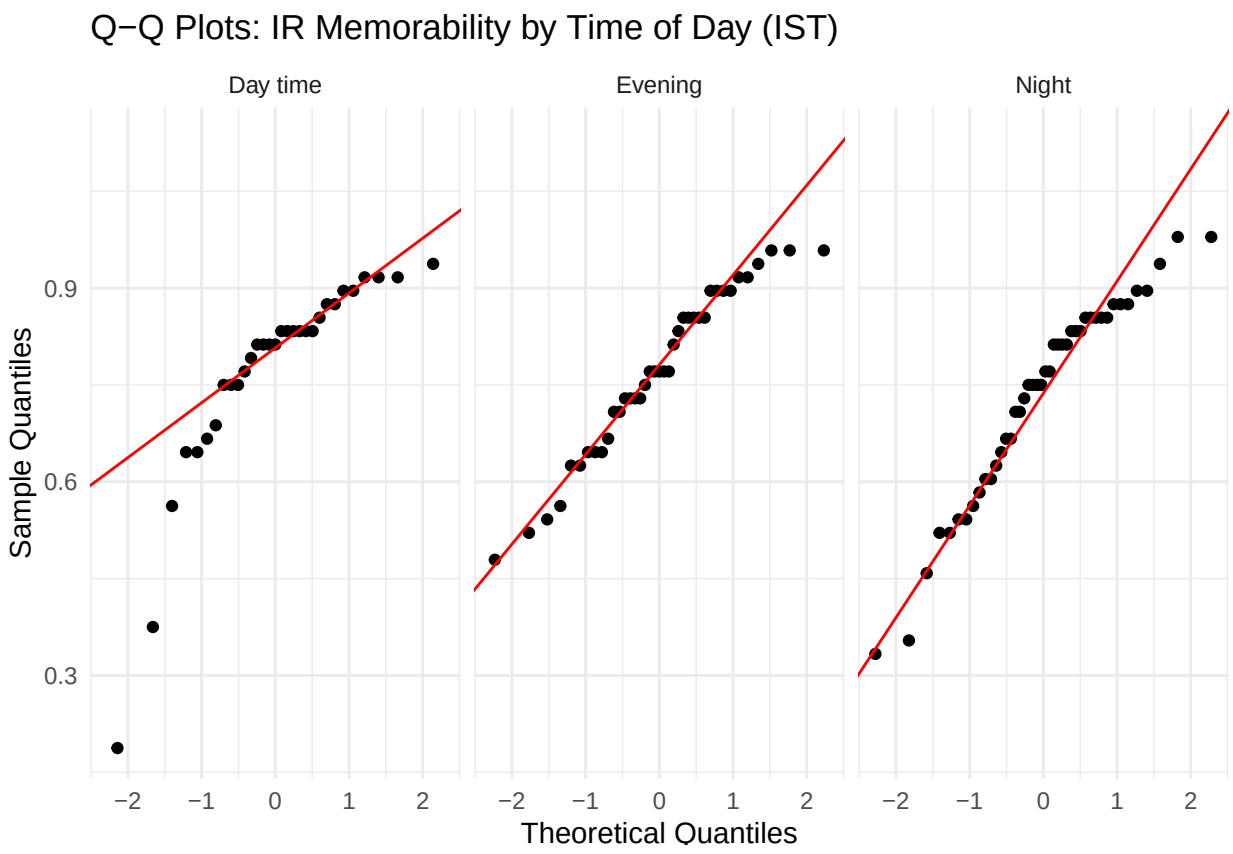
```
## # A tibble: 2 x 3
##   Measure             n_pairs wilcox_r
##   <chr>                 <int>    <dbl>
## 1 H6a_IR_Memorability     114   0.0494
## 2 H6b_WR_Accuracy         114   0.0228
```

```
## # A tibble: 0 x 9
## # i 9 variables: participant_ID <dbl>, Total_Repeats <int>, Novel_Trials <int>,
## #   Hits <int>, False_Alarms <int>, Hit_Rate <dbl>, False_Alarm_Rate <dbl>,
## #   IR_Memorability <dbl>, Sentence_Shown_Count <int>
```

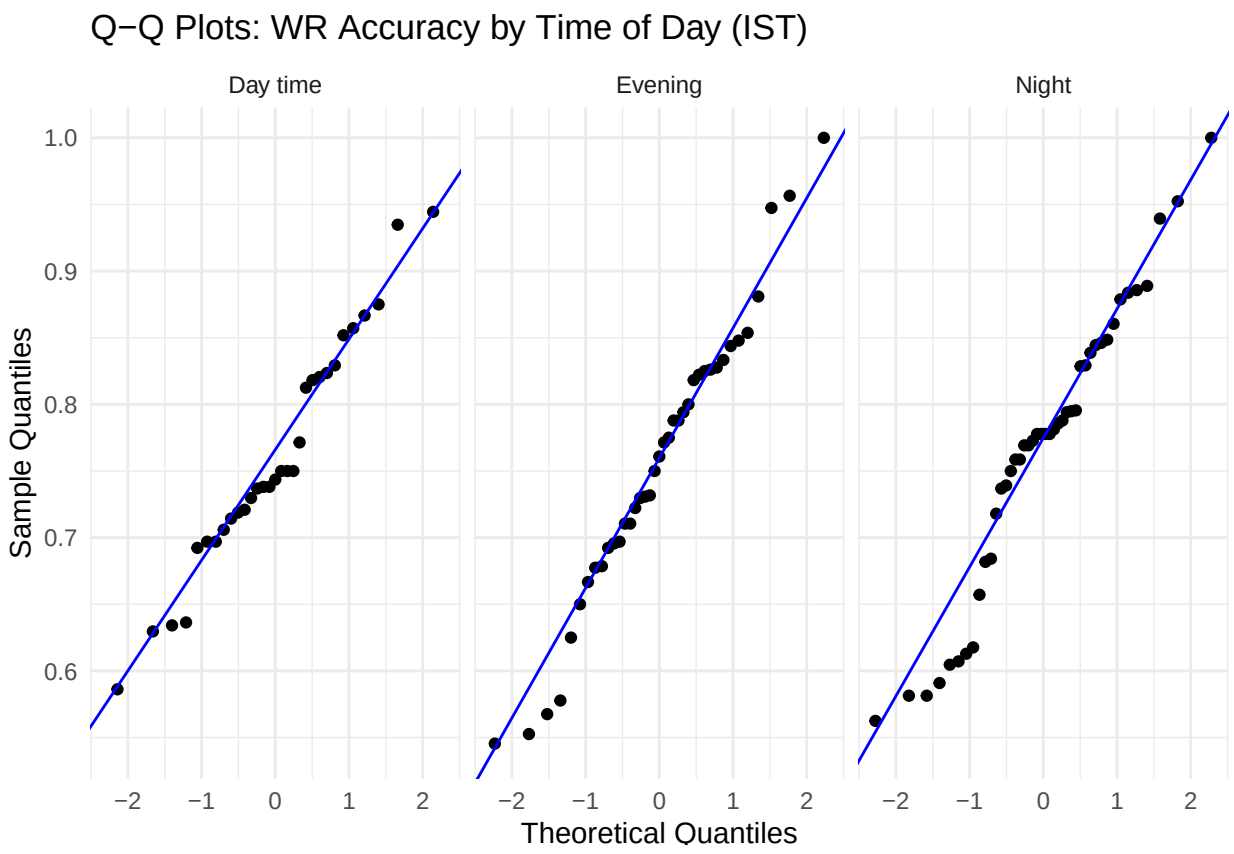
```r
# H7 uses one row per participant from the main experimental phase.
h7_data <- participant_phase_metrics %>%
  filter(Phase == "Main", meets_sentence_threshold) %>%
  drop_na(Time_of_Day)

h7_group_sizes <- h7_data %>%
  count(Time_of_Day)

h7_group_sizes
```

```
## # A tibble: 3 x 2
##   Time_of_Day     n
##   <fct>       <int>
## 1 Day time       31
## 2 Evening        39
## 3 Night          44
```

```r
# Normality checks for H7 measures
ggplot(h7_data, aes(sample = IR_Memorability)) +
  stat_qq() +
  stat_qq_line(color = "red") +
  facet_wrap(~ Time_of_Day, ncol = 3, drop = FALSE) +
  labs(
    title = "Q-Q Plots: IR Memorability by Time of Day (IST)",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_minimal()
```




```
ggplot(h7_data, aes(sample = WR_Accuracy)) +
  stat_qq() +
  stat_qq_line(color = "blue") +
  facet_wrap(~ Time_of_Day, ncol = 3, drop = FALSE) +
  labs(
    title = "Q-Q Plots: WR Accuracy by Time of Day (IST)",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_minimal()
```

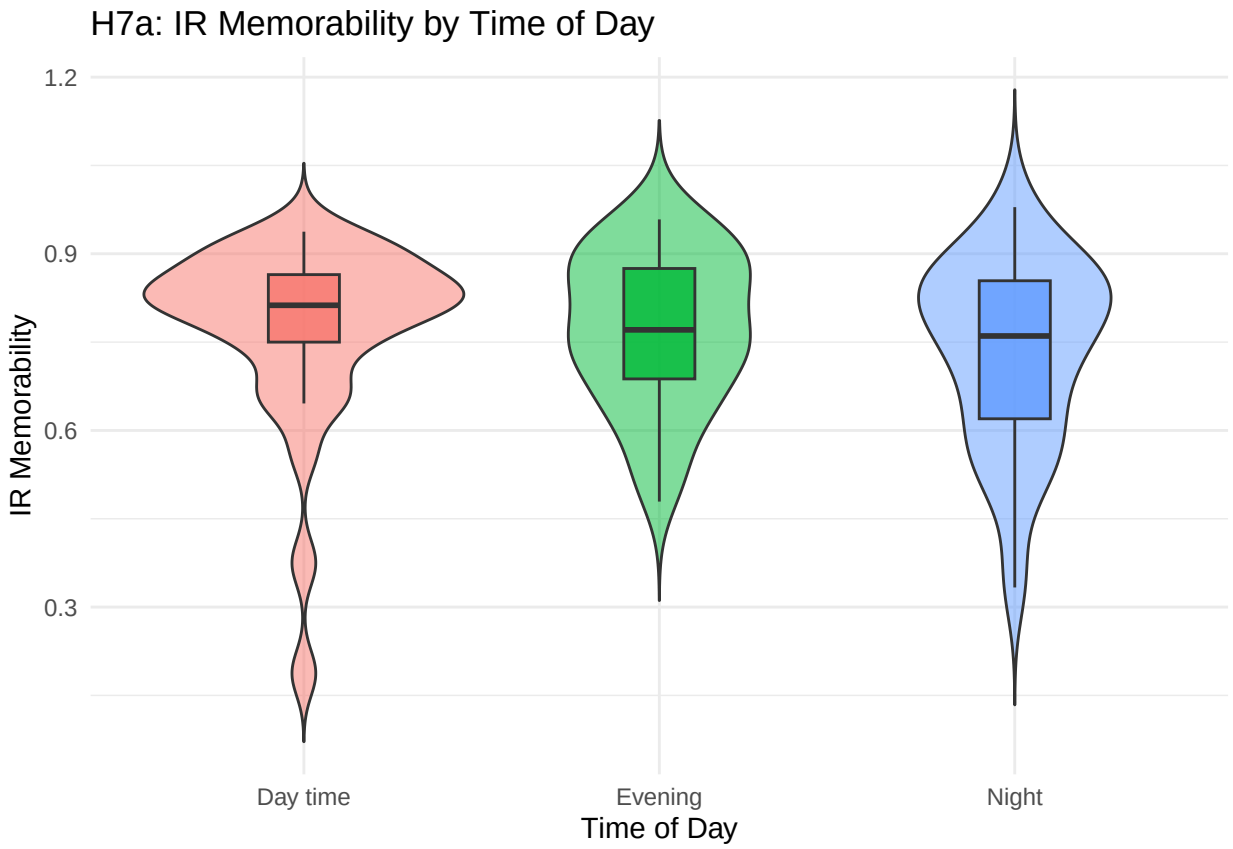



```
h7_data %>%
  group_by(Time_of_Day) %>%
  reframe(
    Measure = c("IR_Memorability", "WR_Accuracy"),
    bind_rows(
      safe_normality_tests(IR_Memorability),
      safe_normality_tests(WR_Accuracy)
    )
  )
```

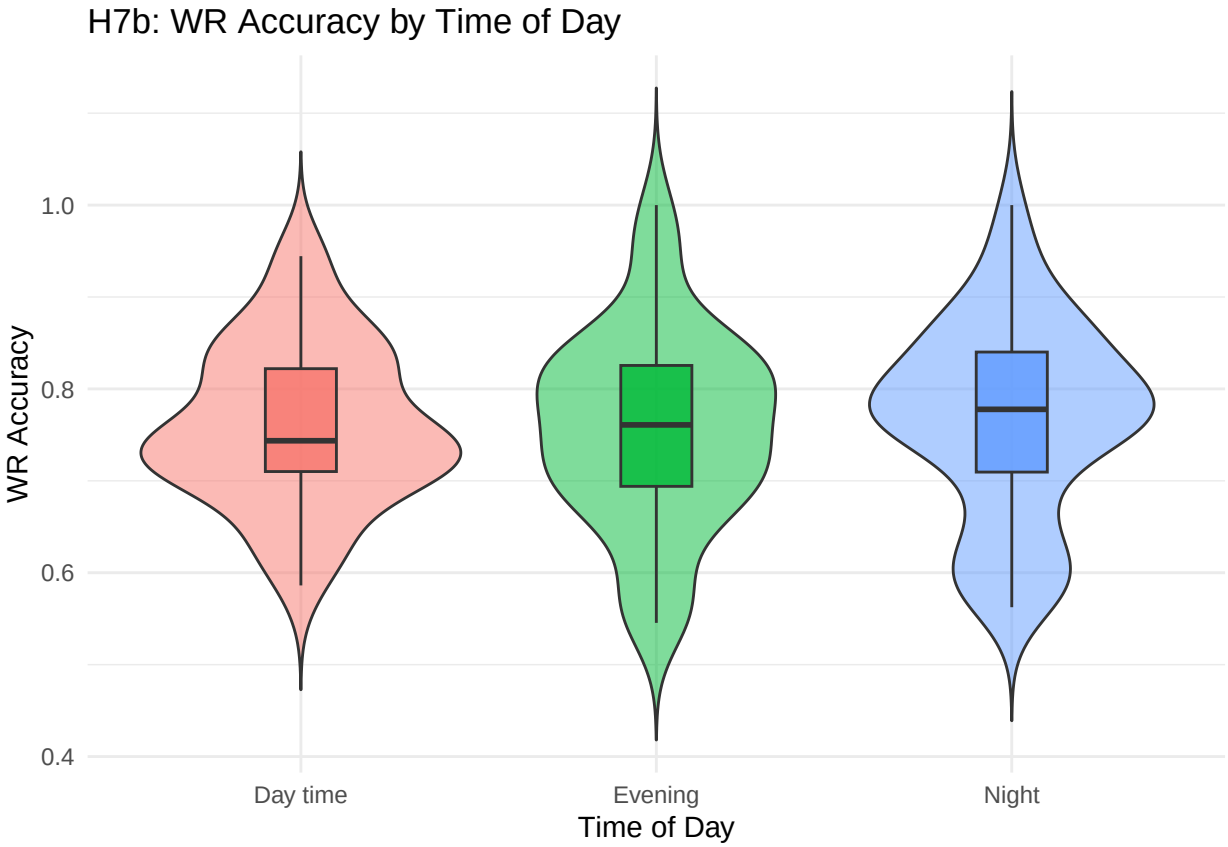
```
## # A tibble: 6 x 6
##   Time_of_Day Measure              n AD_stat      AD_p normality_note
##   <fct>       <chr>            <int>   <dbl>     <dbl> <chr>
## 1 Day time    IR_Memorability     31   2.13  0.0000149 Use QQ + AD jointly
## 2 Day time    WR_Accuracy         31   0.424 0.299     Use QQ + AD jointly
## 3 Evening     IR_Memorability     39   0.489 0.210     Use QQ + AD jointly
## 4 Evening     WR_Accuracy         39   0.255 0.711     Use QQ + AD jointly
## 5 Night       IR_Memorability     44   0.886 0.0215    Use QQ + AD jointly
## 6 Night       WR_Accuracy         44   0.751 0.0468    Use QQ + AD jointly
```

```
# H7a: IR memorability by time of day
ggplot(h7_data, aes(x = Time_of_Day, y = IR_Memorability, fill = Time_of_Day)) +
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  labs(
    title = "H7a: IR Memorability by Time of Day",
```

```
    x = "Time of Day",
    y = "IR Memorability"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```




```
# H7b: WR accuracy by time of day
ggplot(h7_data, aes(x = Time_of_Day, y = WR_Accuracy, fill = Time_of_Day)) +
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  labs(
    title = "H7b: WR Accuracy by Time of Day",
    x = "Time of Day",
    y = "WR Accuracy"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```




```
h7_summary <- h7_data %>%
  group_by(Time_of_Day) %>%
  summarise(
    IR_mean = mean(IR_Memorability, na.rm = TRUE),
    IR_sd = sd(IR_Memorability, na.rm = TRUE),
    IR_median = median(IR_Memorability, na.rm = TRUE),
    WR_mean = mean(WR_Accuracy, na.rm = TRUE),
    WR_sd = sd(WR_Accuracy, na.rm = TRUE),
    WR_median = median(WR_Accuracy, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  )

h7_summary
```

```
## # A tibble: 3 x 8
##   Time_of_Day IR_mean IR_sd IR_median WR_mean  WR_sd WR_median     n
##   <fct>         <dbl> <dbl>     <dbl>   <dbl>  <dbl>     <dbl> <int>
## 1 Day time      0.772 0.161     0.812   0.760 0.0872     0.744    31
## 2 Evening       0.770 0.130     0.771   0.756 0.107      0.761    39
## 3 Night         0.733 0.157     0.760   0.767 0.107      0.778    44
```

```
kruskal_h7_ir <- kruskal.test(IR_Memorability ~ Time_of_Day, data = h7_data)
kruskal_h7_wr <- kruskal.test(WR_Accuracy ~ Time_of_Day, data = h7_data)

h7_effect_sizes <- tibble(
```

```
  Measure = c("H7a_IR_Memorability", "H7b_WR_Accuracy"),
  n_total = nrow(h7_data),
  k_groups = n_distinct(h7_data$Time_of_Day),
  epsilon_sq = c(
    kruskal_epsilon_sq(kruskal_h7_ir, nrow(h7_data), n_distinct(h7_data$Time_of_Day)),
    kruskal_epsilon_sq(kruskal_h7_wr, nrow(h7_data), n_distinct(h7_data$Time_of_Day))
  )
)

print("Kruskal-Wallis test for H7a (IR Memorability):")
```

```
## [1] "Kruskal-Wallis test for H7a (IR Memorability):"
```

```
print(kruskal_h7_ir)
```

```
##
##  Kruskal-Wallis rank sum test
##
## data:  IR_Memorability by Time_of_Day
## Kruskal-Wallis chi-squared = 2.147, df = 2, p-value = 0.3418
```

```
print("Kruskal-Wallis test for H7b (WR Accuracy):")
```

```
## [1] "Kruskal-Wallis test for H7b (WR Accuracy):"
```

```
print(kruskal_h7_wr)
```

```
##
##  Kruskal-Wallis rank sum test
##
## data:  WR_Accuracy by Time_of_Day
## Kruskal-Wallis chi-squared = 0.71746, df = 2, p-value = 0.6986
```

```
print("H7 effect sizes (epsilon squared):")
```

```
## [1] "H7 effect sizes (epsilon squared):"
```

```
print(h7_effect_sizes)
```

```
## # A tibble: 2 x 4
##   Measure             n_total k_groups epsilon_sq
##   <chr>                 <int>    <int>      <dbl>
## 1 H7a_IR_Memorability     114        3    0.00132
## 2 H7b_WR_Accuracy         114        3   -0.0116
```

```
pairwise_h7_ir <- NULL
pairwise_h7_wr <- NULL

if (kruskal_h7_ir$p.value < 0.05) {
```

```r
  pairwise_h7_ir <- pairwise.wilcox.test(
    h7_data$IR_Memorability,
    h7_data$Time_of_Day,
    p.adjust.method = "BH",
    exact = FALSE
  )
  print("Pairwise Wilcoxon tests for H7a (IR Memorability):")
  print(pairwise_h7_ir)
} else {
  print("H7a pairwise tests skipped: Kruskal-Wallis not significant.")
}
```

```
## [1] "H7a pairwise tests skipped: Kruskal-Wallis not significant."
```

```r
if (kruskal_h7_wr$p.value < 0.05) {
  pairwise_h7_wr <- pairwise.wilcox.test(
    h7_data$WR_Accuracy,
    h7_data$Time_of_Day,
    p.adjust.method = "BH",
    exact = FALSE
  )
  print("Pairwise Wilcoxon tests for H7b (WR Accuracy):")
  print(pairwise_h7_wr)
} else {
  print("H7b pairwise tests skipped: Kruskal-Wallis not significant.")
}
```

```
## [1] "H7b pairwise tests skipped: Kruskal-Wallis not significant."
```